

Language Modeling part LAB 4

Christian Dal Farra (mat. 258243)

University of Trento

christian.dalfarra@studenti.unitn.it

1. Introduction

In this report are summarized the experiments done for the first part of the NLU exam project, related to Language Modeling. Starting from a given recurrent neural network and progressively adding architectural and optimization techniques to improve performance, measured in terms of Perplexity on the Penn Treebank dataset. The primary goal is to reduce the PPL below the required threshold (250) while analyzing the effectiveness of each modification. Part A focuses on replacing the vanilla RNN with an LSTM, adding dropout layers, and experimenting with a different optimizer AdamW. Part B explores more advanced regularization strategies, including weight-tying, variational dropout, and Non-monotonically Triggered AvSGD, following insights from the literature [1].

2. Implementation details

In part A, I used the setup for the training given in lab4 $n_epochs = 100$, $clip = 5$, and $patience = 3$. I decided to perform a grid search for the hyperparameters of the network and train with those pools of values $emb_size = [250, 300, 350]$, $hid_size = [250, 300, 350]$, and $lr = [0.001, 0.05, 0.1, 0.5]$. After the search, I decided to set the network's hidden and embedding size both to 350, for the learning rate 0.5 get the best PPL with batch size of 64 (i have tested with 32, 64, 128 and the best results were with 64).

Then I substitute the RNN module with an LSTM one. This new configuration improved performance, so I continued adding the first dropout after the embedding layer starting with a probability of 0.5 then I tried with 0.2 and got similar results with faster training, then added the second dropout before the last layer with a probability of 0.2. Last, I changed the optimizer to AdamW, and this also needs to decrease the learning rate to 0.0002 and this gave the best results for part A.

In part B, I started from the baseline LSTM (without dropouts) and reset the optimizer to SGD with the learning rate set to 0.5, then tried Weight-tying, which sets the weights of the output layer with the values of the weights of the embedding layer. This reduces the number of parameters, leading to faster training, improving efficiency, and regularization, because the model learns to use similar representations in encoding and predicting tokens.

The next step consists of adding the Variational Dropout, a regularization technique that uses the same dropout mask in all timesteps, for the experiments i have used two variational dropout layers, as in part A, after the embeddings and before the last layer, with probability set to 0.2 (tested with 0.1, 0.2, 0.5), then tried with three different learning rate 0.1 0.5 and 1 and with 0.5 i had the best PPL.

The last step was to add the NT-AvSGD, an optimization method where the averaging of model parameters starts only when the validation performance stops improving for some consecutive checks, instead of starting at a

fixed epoch as in the classical AvSGD. This adaptive trigger prevents premature averaging, helping the model stabilize and achieve better generalization. NT-AvSGD removes the need to manually tune the starting point of averaging, for the implementation, i look at the repository of the paper [1] (<https://github.com/salesforce/awd-lstm-lm/blob/master/finetune.py>). For the experiment, the log interval is set at 1 (checks every epoch), the n (non-monotonic interval) to 5, as suggested in the paper, and the patience was automatically set to $n \times 2$, to allow observation of monotonic behavior before early-stopping the training, for the learning rate i tried with 0.5, 1 and 2, with 2 giving the best results.

3. Results

In Part A and Part B the metrics chosen for the evaluation was the Perplexity on the Penn Treebank dataset. In the Table 1 there are the experiments, all under the required threshold of 250 PPL, as we can see every change in the network causes an improvement in performances. in Figure 2 and 1 are plotted respectively the PPL and the training/validation loss for the best configuration of Part A, the training stops at epoch 60 due to the early-stopping regularization method to prevent overfitting

Model	Perplexity
RNN (baseline)	159.83
LSTM	147.94
LSTM+DO	118.61
LSTM+DO+AdamW	113.52

Table 1: Part A results

In Table 2 there are the experiments for the second part, in Figure 4 and 3 there are the plots for the perplexity and loss of the best model in part B, focusing on the loss plot we can see that at epoch 34 the non-monotonic is triggered and the optimizer is changed to ASGD, also the train stops at epoch 58 due to early stopping regularization.

Model	Perplexity
LSTM (baseline)	147.94
LSTM+WT	128.17
LSTM+WT+VarDO	110.75
LSTM+WT+VarDO+NT-AvSGD	106.44

Table 2: Part B results

4. References

- [1] S. Merity, N. S. Keskar, and R. Socher, "Regularizing and optimizing lstm language models," *arXiv preprint arXiv:1708.02182*,

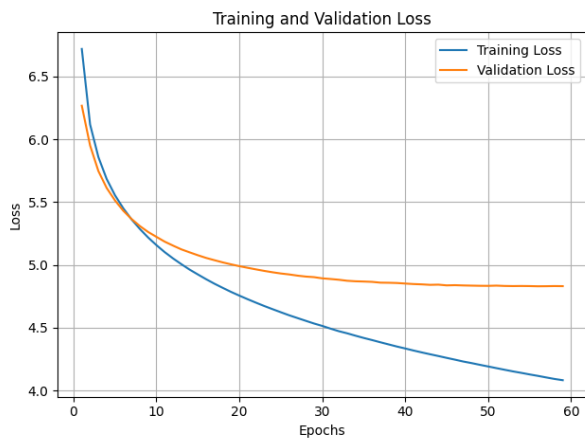


Figure 1: Loss plot for the last experiment of the first part

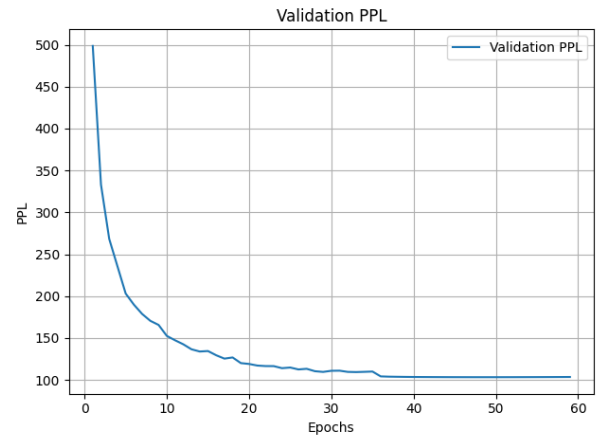


Figure 4: PPL plot for the last experiment of the second part

2017. [Online]. Available: <https://arxiv.org/abs/1708.02182>

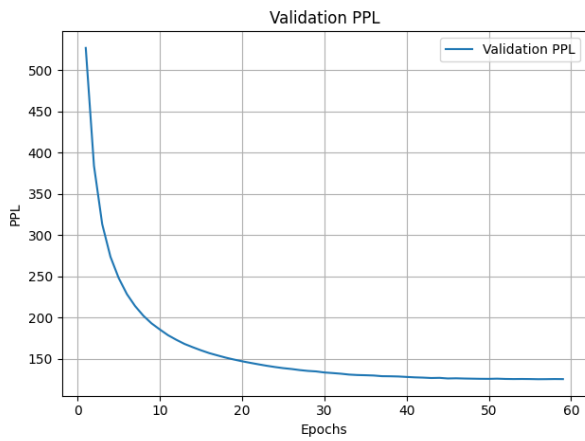


Figure 2: PPL plot for the last experiment of the first part

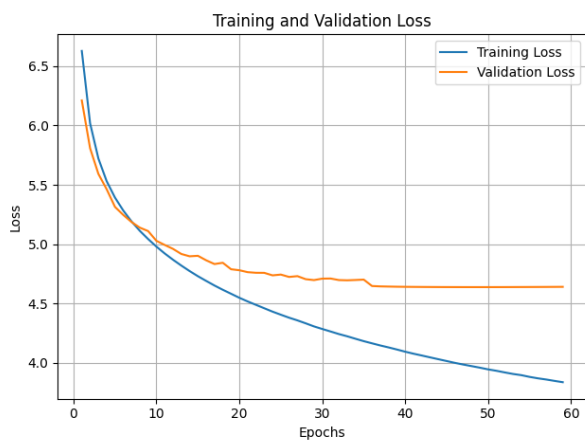


Figure 3: Loss plot for the last experiment of the second part